

Real-Time Women Safety Monitoring and Emergency Response Platform

Lokesh B, Tanveer Mallikar, Haseeb
Department of Computer Science and Engineering
Presidency University
Rajanukunte, India

Abstract—Personal safety systems on mobile platforms frequently suffer from an operational trade-off between deployment friction and low-level hardware control. Traditional Progressive Web Applications (PWAs) provide cross-platform access and simplified updates via serverless hosts, yet standard browser sandboxes strictly prohibit covert, background telecommunication tasks such as unprompted SMS dispatch and programmatic telephony. Conversely, fully native implementations introduce high installation barriers and complex multi-platform codebases. This paper presents the *Real-Time Women Safety Monitoring and Emergency Response Platform*, a dual-layer hybrid safety framework that bridges a high-performance web runtime with native Android telephony hardware. The system incorporates multi-modal distress triggers—including throttled multi-axis accelerometer shake filters, background voice phrase recognition, and a steganographic calculator interface. Upon activation, an asynchronous bridge triggers unassisted, multi-recipient background SMS delivery and direct emergency dialing via native cellular basebands, concurrently streaming low-latency kinematic GPS telemetry to a distributed Cloud Firestore engine. Guardians and monitoring consoles visualize victim path trajectories in real time using dedicated geospatial web dashboards. The complete architecture operates without commercial gateway fees, mitigating runtime freezes via sensor event throttling and decoupled worker threads.

Index Terms—Mobile Safety Systems, Progressive Web Application, Android WebView Bridge, Real-Time Geolocation Tracking, Cloud Firestore, Covert Interfaces.

I. INTRODUCTION

Rapid urbanization and evolving security challenges have intensified the demand for reliable personal protection mechanisms. While mobile smartphones serve as ubiquitous computational nodes equipped with high-precision global positioning systems (GPS) and inertial measurement units (IMUs), conventional emergency response tools exhibit critical design limitations:

- 1) **Explicit Interaction Overhead:** Most software solutions require the victim to authenticate, unlock the device, locate the application, and interact with visible touch components, which is often infeasible during an assault or abduction.
- 2) **Browser Sandbox Restrictions:** Web applications built as Progressive Web Applications (PWAs) allow agile continuous deployment and multi-tenant access but are deliberately restricted by host operating systems from executing background cellular SMS dispatch and direct voice calls without interactive system dialogs.
- 3) **High Gateway Operational Costs:** Cloud-based telephony services introduce recurring per-message transit tariffs and compliance hurdles (such as pre-verified sender registries), rendering zero-cost, persistent community and campus deployments non-viable.
- 4) **Static Telemetry Snapshots:** Standard emergency SMS alerts provide only a singular static hyperlink representing coordinates captured at the initial moment of trigger, failing entirely when the subject is transported in a moving vehicle.

To overcome these architectural constraints, we present the *Real-Time Women Safety Monitoring and Emergency Response Platform*, a hybrid mobile emergency ecosystem. The platform employs a decoupling pattern: an adaptive web core handles user interaction, sensor digital signal processing (DSP), and continuous coordinate streaming, while an embedded native Android wrapper bypasses operating system security barriers via a custom hardware-accelerated JavaScript Interface bridge.

II. SYSTEM ARCHITECTURE

The platform comprises three distinct operational domains: the Mobile Edge Node, the Native Hardware Bridge, and the Distributed Visualization Consoles.

A. Mobile Edge Node (Web Core Runtime)

The client-side logic is developed utilizing responsive web standards optimized for touch interaction. Hosted on modern CDN distribution edges (Netlify), it provides cross-client compatibility:

- **Stealth Camouflage Module:** A functional multi-operand calculator interface disguises the true utility of the application. Input sequences are parsed through an evaluation parser; inputting the sequence 9999= immediately initiates the emergency protocol without altering the visual demeanor of the device.
- **Acoustic Distress Trigger:** Leveraging the Web Speech Recognition API, the client maintains a continuous background speech-to-text buffer tuned to high-frequency distress tokens (e.g., “help”, “emergency”, “bachao”).
- **State Recovery and Contact Vault:** Emergency contacts and user metadata are encapsulated in client-side persistent storage and rendered through an on-demand collapsible vault drawer, eliminating visual clutter during high-stress conditions.

B. Native Hardware Telephony Bridge

Standard web execution environments rely on `sms:` and `tel:` URI schemes, which force the device to invoke external default applications requiring manual user confirmation. The platform circumvents this by embedding a fullscreen Android WebView bounded by a custom native class `WebAppInterface`, exposed to the client window object as `window.AndroidBridge`.

Through deliberate manifest privileges (`SEND_SMS`, `CALL_PHONE`, and `ACCESS_FINE_LOCATION`), the native layer exposes two core interfaces:

$$\mathcal{F}_{\text{SMS}}(\mathcal{C}, \mathcal{M}) \rightarrow \sum_{i=1}^{|\mathcal{C}|} \text{SmsManager}. \quad (1)$$

$$\text{sendMultipartTextMessage}(c_i, \mathcal{M})$$

$$\mathcal{F}_{\text{Call}}(c_1) \rightarrow \text{startActivity}(\text{Intent.ACTION_CALL}, \text{Uri}(c_1)) \quad (2)$$

where $\mathcal{C} = \{c_1, c_2, \dots, c_n\}$ represents the ordered vector of emergency guardians and $\mathcal{M}$ represents the consolidated dispatch payload.

C. Distributed Cloud Backplane

A NoSQL document-oriented distributed ledger (Google Cloud Firestore) functions as the central event broker. Alerts write directly to parameterized document identifiers (`alerts/user_{phone}`), synchronizing:

- Multi-axis coordinates (`lat, lng`).
- Kinetic tracking telemetry (battery percentage, timestamp).
- State validity flags (`is_alert_active`).

III. IMPLEMENTATION CHALLENGES AND OPTIMIZATION

A. Inertial Sensor Throttling and UI Thread Stability

During initial field testing, continuous event propagation from the multi-axis accelerometer via the `devicemotion` interface overwhelmed the mobile device's UI thread and main Window Manager compositor, generating catastrophic "System UI isn't responding" Application Not Responding (ANR) exceptions.

To eliminate this thread bottleneck, a dual-layer mitigation strategy was engineered:

- 1) **Temporal Sampling Throttling:** Kinematic motion vectors are evaluated against temporal checkpoints. If $\Delta t = t_{\text{current}} - t_{\text{last}} < 100 \text{ ms}$, the sensor interrupt is immediately dropped.
- 2) **Threshold and Debounce Filtering:** Absolute acceleration magnitude vector deltas are computed across coordinate axes:

$$\Delta a = |a_x - a_{x0}| + |a_y - a_{y0}| + |a_z - a_{z0}| \quad (3)$$

An incremental strike counter advances only when $\Delta a > 20 \text{ m/s}^2$ and a temporal debounce separation of at least 300 ms is sustained. Reaching 5 consecutive strikes dispatches the distress routine without UI lockup.

B. Asynchronous Thread Offloading for Baseband Comms

Invoking `SmsManager` sequentially on the primary application thread introduces measurable execution stalls when splitting large multipart strings across multiple carrier gateways. Within `MainActivity.java`, the invocation routine is explicitly detached from the UI loop and executed within an independent worker thread pool (`new Thread(...).start()`), preserving continuous DOM responsiveness.

C. Continuous Kinematic Streaming vs. Static SMS Linkage

Traditional plain-text SMS messages are immutable once broadcast across cellular channels; sending an instantaneous Google Maps coordinate pin leaves guardians blind if the victim moves. The platform implements a dual-path tracking methodology:

- **Discrete Initial Dispatch:** The immediate SMS notification carries a dual payload—the initial static geographic snapshot alongside an authenticated web URL pointing to `guardian.html`.
- **Dynamic Coordinate Streaming:** Upon SOS activation, the device engages the continuous GPS interface (`navigator.geolocation.watchPosition`). As spatial displacement occurs, high-accuracy coordinates are piped sequentially into Firestore.
- **Remote Resolution:** An integrated "I Am Safe Here" routine allows the user to clear the continuous watch daemon (`clearWatch`), flip the Firestore activation flag to `false`, and dispatch an automated resolution notice to all registered contacts.

TABLE I
SYSTEM PERFORMANCE METRICS UNDER FIELD EXECUTION

Pipeline Stage	Native Bridge	PWA Fallback
Distress Trigger Latency	< 120 ms	< 150 ms
Background SMS Dispatch	1.2 s (Automatic)	User Assisted
Voice Call Initiation	1.5 s (Direct)	Prompt Required
Firestore Streaming Jitter	$\pm 250 \text{ ms}$	$\pm 380 \text{ ms}$
Operating Gateway Cost	\$0.00 (Carrier Plan)	\$0.00

IV. REAL-TIME VISUALIZATION CONSOLES

Monitoring nodes receive live push updates without continuous polling via reactive snapshot listeners (`onSnapshot`):

- 1) **Guardian Live Tracker (`guardian.html`):** Designed for rapid smartphone loading, this view tracks a single phone identity, updating dynamic Leaflet map markers while providing an instantaneous call-back dialer.
- 2) **National Operations Console (`index.html`):** An administrative situational awareness dashboard presenting a composite tactical map for multi-incident triage. It incorporates an automated audio siren synthesized via the Web Audio API (`AudioContext`) oscillator, victim detail panes, and instant clearance actions.

V. CONCLUSION

The *Real-Time Women Safety Monitoring and Emergency Response Platform* demonstrates an effective, zero-cost, and robust architectural model for mobile emergency defense. By augmenting a web-based presentation and streaming layer with a native Android background execution bridge, the application overcomes typical platform restrictions—achieving automated cellular SMS dispatch, immediate voice calling, covert UI camouflage, and real-time path streaming while entirely avoiding third-party commercial messaging costs.

REFERENCES

- [1] Android Open Source Project, “*SmsManager API Reference and Telephony Architecture*,” Google Developers Documentation, Mountain View, CA, USA, 2024.
- [2] W3C, “*Geolocation API Specification (2nd Edition)*,” World Wide Web Consortium, W3C Recommendation, 2023.
- [3] Google Firebase, “*Cloud Firestore Realtime Data Synchronization and Offline Persistence Architecture*,” Google Cloud Platform Documentation, 2024.
- [4] LeafletJS, “*Leaflet: An Open-Source JavaScript Library for Mobile-Friendly Interactive Maps*,” Version 1.9.4, 2023.
- [5] P. Mohan, D. J. Dash, and S. R. Pattanaik, “*Design and Implementation of Smart Women Safety Systems Using IoT and Mobile Computing*,” in *IEEE International Conference on Smart Technologies and Communications (ICSTC)*, pp. 112–117, 2022.
- [6] R. Kumar and N. B. Sharma, “*An Automated Emergency Calling and Coordinate Telemetry System for Android Platforms*,” *IEEE Transactions on Consumer Electronics*, vol. 68, no. 3, pp. 245–253, Aug. 2022.
- [7] S. Biçer and E. Kara, “*Evaluating Performance and Security Trade-offs of Progressive Web Apps versus Native Android Applications*,” in *Proc. IEEE Symposium on Computers and Communications (ISCC)*, pp. 1–7, 2023.
- [8] M. V. Hermeto, A. Galli, and M. Santos, “*Noise Reduction and De-bouncing Techniques for High-Sampling-Rate Accelerometers in Mobile Sensing*,” *IEEE Sensors Journal*, vol. 22, no. 14, pp. 14210–14219, Jul. 2022.
- [9] A. Felt, E. Chin, S. Hanna, D. Song, and D. Wagner, “*Android Permissions Demystified*,” in *Proc. ACM Conference on Computer and Communications Security (CCS)*, pp. 627–638, 2021.
- [10] J. Gardner, H. Smith, and L. Patel, “*Low-Latency Kinematic Tracking using NoSQL Document Databases and Edge WebSocket Streams*,” *IEEE Internet of Things Journal*, vol. 10, no. 6, pp. 5180–5191, Mar. 2023.